

PatientList.svelte : Patient list management component

Project reference: STN22_Thoracic belt for respiratory rate measurement

Authors: Hakkou Dounia

Date: 2025-2026 © SIS TEAM NANCY

File header

```
<!--
Description: Svelte component for patient list management.
Fetches all patients from the REST API on mount and renders them as cards.
Each card exposes three actions: select for measurement, edit, and delete.
Dispatches 'selectPatient' and 'editPatient' events to the parent (App.svelte).
A global click listener closes any open dropdown menu when the user
clicks outside the menu container.

Project reference: STN22_Thoracic belt FR
Authors: Hakkou Dounia
Date: 2025-2026
© SIS TEAM NANCY
-->
```

Script block (TypeScript)

```
<script lang="ts">
  // Import patient CRUD helpers and the Patient type from the shared API module
  import { patientsAPI, type Patient } from './api';
  // createEventDispatcher: Svelte utility to emit typed custom events to the parent
  // onMount: lifecycle hook called once the component is inserted into the DOM
  import { createEventDispatcher, onMount } from 'svelte';

  // Instantiate the event dispatcher with typed event payloads:
  // - 'selectPatient': emitted when the user clicks 'Mesurer' on a card
  // - 'editPatient' : emitted when the user selects 'Modifier' in the dropdown
  const dispatch = createEventDispatcher<{
    selectPatient: Patient;
    editPatient: Patient;
 }>();

  // Reactive array holding the list of patients returned by the API
  let patients: Patient[] = [];
  // Boolean flag: true while the API request is in progress
  let loading = true;
  // ID of the patient whose dropdown menu is currently open; null if all closed
  let openMenuId: string | null = null;

  // loadPatients
  // Exported so the parent component can trigger a reload after adding a patient.
  // Sets loading=true, calls GET /patients, updates the patients array.
  export async function loadPatients() {
    loading = true;
    try {
      // Fetch all patients from the backend REST API
      patients = await patientsAPI.getAll();
    } catch (err) {
      console.error('Erreur lors du chargement des patients:', err);
      patients = []; // Reset to empty list on error to avoid stale data
    } finally {
      loading = false; // Always stop loading indicator, even on error
    }
  }

  // selectPatient
  // Dispatches the 'selectPatient' event with the chosen patient as payload.
  // The parent receives this event and sets the active patient.
  function selectPatient(patient: Patient) {
    dispatch('selectPatient', patient);
  }

  // toggleMenu
  // Opens or closes the three-dot dropdown menu for a given patient card.
```

```

// stopPropagation prevents the global click listener from immediately
// re-closing the menu that was just opened.
function toggleMenu(event: Event, patientId: string) {
  event.stopPropagation();
  openMenuId = openMenuId === patientId ? null : patientId;
}

// closeMenus
// Global click handler attached to <svelte:window>.
// Closes any open dropdown when the user clicks outside a .menu-container.
function closeMenus(event: Event) {
  const target = event.target as HTMLElement | null;
  if (!target?.closest('.menu-container')) {
    openMenuId = null;
  }
}

// deletePatient
// Asks for confirmation before permanently deleting a patient and all their
// associated sessions and measurements from the database.
async function deletePatient(patient: Patient) {
  // Native browser confirm dialog user must explicitly approve deletion
  const ok = confirm(
    `Etes-vous sur de vouloir supprimer le patient ${patient.prenom} ${patient.nom}
? \n \n` +
    `Cette action supprimera également toutes ses sessions et mesures.`
  );
  if (!ok) { openMenuId = null; return; } // Cancelled: close menu and abort

  try {
    // DELETE /patients/:id, removes the patient record and cascades to
    sessions/measurements
    await patientsAPI.delete(patient.id);
    alert('Patient supprime avec succes');
    await loadPatients(); // Refresh the list to reflect the deletion
  } catch (err) {
    console.error('Erreur lors de la suppression:', err);
    alert('Erreur lors de la suppression du patient');
  } finally {
    openMenuId = null; // Close dropdown regardless of outcome
  }
}

// editPatient
// Dispatches the 'editPatient' event with the patient to edit as payload.
// The parent opens PatientForm.svelte pre-filled with this patient's data.
function editPatient(patient: Patient) {
  dispatch('editPatient', patient);
  openMenuId = null; // Close the dropdown after dispatching
}

// onMount
// Svelte calls this once the component is in the DOM.
// Triggers the initial patient list fetch so cards are displayed immediately.
onMount(loadPatients);
</script>

```

HTML template (Svelte)

```

<!-- Attach global click listener to close any open dropdown menu -->
<svelte:window on:click={closeMenus} />

<div class="patient-list">
  <div class="header">
    <h2>Patients</h2>
    <!-- Refresh button: reloads the patient list from the API -->
    <!-- Disabled while a fetch is already in progress (loading=true) -->
    <button class="refresh-btn" on:click={loadPatients} disabled={loading}>
      Actualiser
    </button>
  </div>

  {#if loading}

```

```

<!-- Shown while the API request is pending -->
<div class="loading">Chargement...</div>

{:else if patients.length === 0}
  <!-- Shown when the API returns an empty array -->
  <div class="empty">Aucun patient trouve</div>

{:else}
  <!-- Grid of patient cards, keyed by patient.id for efficient DOM diffing -->
  <div class="patients-grid">
    {#each patients as patient (patient.id)}
      <div class="patient-card">
        <div class="card-header">
          <div class="patient-info">
            <!-- Patient identity and anthropometric details -->
            <h3>{patient.prenom} {patient.nom}</h3>
            <div class="details">
              <span>{patient.age} ans</span>
              <span>{patient.sexe}</span>
              <span>{patient.poids} kg</span>
              <span>{patient.taille} cm</span>
            </div>
            <!-- Optional phone number (nullable field in DB) -->
            <div class="contact">{patient.telephone ?? ''}</div>
          </div>

          <!-- Three-dot dropdown menu container -->
          <div class="menu-container">
            <!-- Toggle button: opens/closes the dropdown for this card -->
            <button class="menu-button"
              on:click={ (e) => toggleMenu(e, patient.id)}
              aria-label="Options">
              :
            </button>

            <!-- Dropdown rendered only when this patient's menu is open -->
            {#if openMenuId === patient.id}
              <div class="menu-dropdown" role="menu" tabindex="-1"
                on:click|stopPropagation on:keydown|stopPropagation>
                <!-- Edit: opens PatientForm pre-filled with this patient -->
                <button class="menu-item edit"
                  on:click={ () => editPatient(patient)} role="menuitem">
                  Modifier
                </button>
                <!-- Delete: triggers confirmation dialog then API call -->
                <button class="menu-item delete"
                  on:click={ () => deletePatient(patient)} role="menuitem">
                  Supprimer
                </button>
              </div>
            {/if}
          </div>

          <!-- Measure button: selects this patient and navigates to the measure tab -->
          <div class="actions">
            <button class="measure-btn" on:click={ () => selectPatient(patient)}>
              Mesurer
            </button>
          </div>
        </div>
      {/each}
    </div>
  {/if}
</div>

```

CSS styles (scoped)

```

<style>
  /* Scoped to this component – styles do not leak to parent or siblings */

  /* Main container: white card with subtle shadow */
  .patient-list { background: white; padding: 2rem; border-radius: 12px;

```

```
    box-shadow: 0 2px 8px rgba(0,0,0,0.1); }

/* Responsive grid: auto-fills with minimum 300px wide cards */
.patients-grid { display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap: 1rem; }

/* Card: light purple-to-blue gradient with hover lift animation */
.patient-card { background: linear-gradient(135deg, #faf5ff 0%, #eff6ff 100%);
  padding: 1.5rem; border-radius: 12px; border: 2px solid #e9d5ff;
  transition: all 0.3s; }
.patient-card:hover { transform: translateY(-4px);
  box-shadow: 0 8px 16px rgba(124,58,237,0.15); }

/* Dropdown menu: absolute positioning relative to .menu-container */
.menu-dropdown { position: absolute; right: 0; top: 100%;
  background: white; border-radius: 8px; box-shadow: 0 4px 12px rgba(0,0,0,0.2);
  min-width: 150px; z-index: 1000; }

/* Delete item: red color to signal destructive action */
.menu-item.delete { color: #dc2626; }
.menu-item.delete:hover { background-color: #fee2e2; }

/* Measure button: purple gradient matching the app primary color */
.measure-btn { background: linear-gradient(135deg, #7c3aed 0%, #93c5fd 100%);
  color: white; }
</style>
```